

# Анализ данных: экзаменационный конспект продвинутого уровня

Вероятность, статистика, pandas, ML, тексты, изображения, нейросети

Учебный справочник для экзамена

2026-05-27

## Содержание

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>1 Как пользоваться на экзамене</b>                                               | <b>3</b>  |
| <b>2 1. Вероятность</b>                                                             | <b>3</b>  |
| 2.1 1.1. Базовые понятия . . . . .                                                  | 3         |
| 2.2 1.2. Сложение и умножение вероятностей . . . . .                                | 4         |
| 2.3 1.3. Формула полной вероятности и Байес . . . . .                               | 4         |
| 2.4 1.4. Математическое ожидание и дисперсия . . . . .                              | 5         |
| <b>3 2. Основные распределения</b>                                                  | <b>5</b>  |
| 3.1 2.1. Бернулли . . . . .                                                         | 5         |
| 3.2 2.2. Биномиальное распределение . . . . .                                       | 5         |
| 3.3 2.3. Геометрическое распределение . . . . .                                     | 6         |
| 3.4 2.4. Экспоненциальное распределение . . . . .                                   | 6         |
| 3.5 2.5. Нормальное распределение . . . . .                                         | 7         |
| 3.6 2.6. Как пользоваться таблицей стандартного нормального распределения . . . . . | 7         |
| <b>4 3. Статистика</b>                                                              | <b>8</b>  |
| 4.1 3.1. Генеральная совокупность и выборка . . . . .                               | 8         |
| 4.2 3.2. Типы переменных . . . . .                                                  | 8         |
| 4.3 3.3. Меры центра и разброса . . . . .                                           | 8         |
| 4.4 3.4. Частотные таблицы и распределения . . . . .                                | 9         |
| <b>5 4. Проверка гипотез</b>                                                        | <b>9</b>  |
| 5.1 4.1. Нулевая и альтернативная гипотезы . . . . .                                | 9         |
| 5.2 4.2. p-value и уровень значимости . . . . .                                     | 9         |
| 5.3 4.3. Ошибки первого и второго рода . . . . .                                    | 9         |
| <b>6 5. Корреляция</b>                                                              | <b>10</b> |
| 6.1 5.1. Ковариация и корреляция . . . . .                                          | 10        |
| 6.2 5.2. Типовые ловушки . . . . .                                                  | 10        |
| <b>7 6. Визуализация</b>                                                            | <b>10</b> |
| 7.1 6.1. Как выбрать график . . . . .                                               | 10        |
| 7.2 6.2. Как читать boxplot . . . . .                                               | 10        |
| 7.3 6.3. Как читать violinplot . . . . .                                            | 11        |
| 7.4 6.4. Как читать density и CDF . . . . .                                         | 11        |
| <b>8 7. Работа с таблицами в pandas</b>                                             | <b>11</b> |
| 8.1 7.1. Первичный осмотр данных . . . . .                                          | 11        |
| 8.2 7.2. Фильтрация и выбор столбцов . . . . .                                      | 12        |
| 8.3 7.3. Создание и изменение столбцов . . . . .                                    | 12        |
| 8.4 7.4. Пропущенные значения . . . . .                                             | 12        |
| 8.5 7.5. Изменение типов . . . . .                                                  | 13        |
| 8.6 7.6. Агрегации . . . . .                                                        | 13        |
| 8.7 7.7. Корреляции в pandas . . . . .                                              | 13        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>9 8. Машинное обучение: базовая схема</b>           | <b>14</b> |
| 9.1 8.1. Объекты, признаки, целевая переменная         | 14        |
| 9.2 8.2. Обучение с учителем и без учителя             | 14        |
| 9.3 8.3. Train/test split                              | 14        |
| <b>109. Метрики качества</b>                           | <b>14</b> |
| 10.19.1. Метрики регрессии                             | 14        |
| 10.29.2. Confusion matrix                              | 15        |
| 10.39.3. Accuracy                                      | 16        |
| 10.49.4. Precision, recall, F1                         | 16        |
| 10.59.5. ROC-AUC                                       | 16        |
| 10.69.6. Метрики в sklearn                             | 16        |
| <b>1110. Переобучение, недообучение, регуляризация</b> | <b>17</b> |
| 11.110.1. Недообучение                                 | 17        |
| 11.210.2. Переобучение                                 | 17        |
| 11.310.3. Когда нельзя сделать вывод                   | 18        |
| 11.410.4. Регуляризация                                | 18        |
| <b>1211. Кросс-валидация и подбор гиперпараметров</b>  | <b>18</b> |
| 12.111.1. Зачем нужна кросс-валидация                  | 18        |
| 12.211.2. GridSearchCV                                 | 18        |
| <b>1312. Линейная алгебра для анализа данных и ML</b>  | <b>19</b> |
| 13.112.1. Векторы и матрицы                            | 19        |
| 13.212.2. Скалярное произведение                       | 19        |
| 13.312.3. Норма                                        | 20        |
| 13.412.4. Умножение матриц                             | 20        |
| 13.512.5. Обратная матрица и ранг                      | 20        |
| <b>1413. Линейные модели и градиентный спуск</b>       | <b>20</b> |
| 14.113.1. Линейная регрессия                           | 20        |
| 14.213.2. Логистическая регрессия                      | 21        |
| 14.313.3. Градиентный спуск                            | 21        |
| 14.413.4. Momentum                                     | 21        |
| 14.513.5. AdaGrad и разные шаги для разных весов       | 21        |
| <b>1514. Классические алгоритмы ML</b>                 | <b>22</b> |
| 15.114.1. kNN                                          | 22        |
| 15.214.2. Решающие деревья                             | 22        |
| 15.314.3. Bagging и Random Forest                      | 22        |
| 15.414.4. Boosting                                     | 22        |
| 15.514.5. Bias-variance-noise decomposition            | 23        |
| <b>1615. Кластеризация</b>                             | <b>23</b> |
| 16.115.1. K-means                                      | 23        |
| 16.215.2. DBSCAN                                       | 23        |
| 16.315.3. Метрики кластеризации                        | 24        |
| <b>1716. Тексты</b>                                    | <b>24</b> |
| 17.116.1. Tokenization                                 | 24        |
| 17.216.2. Stop-words                                   | 24        |
| 17.316.3. Stemming и lemmatization                     | 24        |
| 17.416.4. Bag of Words и TF-IDF                        | 25        |
| <b>1817. Изображения</b>                               | <b>25</b> |
| 18.117.1. Представление изображений                    | 25        |
| 18.217.2. Зачем нужна нормализация изображений         | 25        |
| 18.317.3. Базовые операции                             | 25        |
| <b>1918. Основы нейронных сетей</b>                    | <b>26</b> |
| 19.118.1. Нейрон и слой                                | 26        |
| 19.218.2. Dense layer                                  | 26        |
| 19.318.3. Convolutional layer                          | 26        |
| 19.418.4. Pooling                                      | 26        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| 19.518.5. Dropout . . . . .                                          | 26        |
| 19.618.6. Batch normalization . . . . .                              | 26        |
| 19.718.7. Функции активации . . . . .                                | 26        |
| <b>2019. Кодирование категориальных признаков и sklearn Pipeline</b> | <b>27</b> |
| 20.119.1. One-hot encoding . . . . .                                 | 27        |
| 20.219.2. Полный шаблон для задачи классификации . . . . .           | 27        |
| 20.319.3. Полный шаблон для задачи регрессии . . . . .               | 28        |
| 20.419.4. GridSearchCV с Pipeline . . . . .                          | 29        |
| <b>2120. Быстрые экзаменационные правила</b>                         | <b>29</b> |
| 21.120.1. Как распознать целевую переменную . . . . .                | 29        |
| 21.220.2. Как распознать метрику . . . . .                           | 29        |
| 21.320.3. Как распознать ошибку в гипотезах . . . . .                | 29        |
| 21.420.4. Как распознать переобучение . . . . .                      | 29        |
| 21.520.5. Как думать про интервалы вероятности . . . . .             | 30        |
| 21.620.6. Мини-таблица «что выбрать» . . . . .                       | 30        |
| <b>2221. Мини-задачи для самопроверки</b>                            | <b>30</b> |
| 22.121.1. Геометрическое распределение . . . . .                     | 30        |
| 22.221.2. p-value . . . . .                                          | 30        |
| 22.321.3. Accuracy и дисбаланс . . . . .                             | 30        |
| 22.421.4. ROC-AUC константного алгоритма . . . . .                   | 30        |
| 22.521.5. Нормальная таблица . . . . .                               | 30        |
| 22.621.6. Экспоненциальное распределение . . . . .                   | 31        |
| <b>2322. Финальный чек-лист перед сдачей</b>                         | <b>31</b> |
| <b>2423. Краткий словарь</b>                                         | <b>31</b> |

## 1 Как пользоваться на экзамене

Этот конспект сделан как практический справочник. Его удобно держать открытым в PDF и искать по нему через Ctrl+F. Основные запросы для поиска: Bayes, p-value, ROC-AUC, accuracy, MAE, groupby, GridSearchCV, лемматизация, AdaGrad, bias-variance, DBSCAN.

На экзамене обычно не нужно доказывать теоремы. Нужно быстро распознать тип задачи, выбрать формулу, понять, что является объектом, признаком, целевой переменной, метрикой и корректной интерпретацией результата.

Главное правило: сначала определи тип задачи.

| Ситуация                                          | Тип задачи           | Типичный ответ                           |
|---------------------------------------------------|----------------------|------------------------------------------|
| Предсказать число: цену, длительность, количество | Регрессия            | MAE, MSE, RMSE, R2                       |
| Предсказать класс: да/нет, категория, тип клиента | Классификация        | accuracy, precision, recall, F1, ROC-AUC |
| Найти группы без заранее известных ответов        | Кластеризация        | k-means, DBSCAN, silhouette              |
| Есть у / правильные ответы                        | Обучение с учителем  | regression/classification                |
| Нет у, ищем структуру в данных                    | Обучение без учителя | clustering, dimensionality reduction     |

## 2 1. Вероятность

### 2.1 1.1. Базовые понятия

Случайная величина  $X$  принимает значения в зависимости от случайного исхода. Событие - это условие на исход, например  $X \leq 3$  или  $X \geq 5$ .

Вероятность события  $A$  обозначается  $P(A)$  и лежит в интервале:

$$0 \leq P(A) \leq 1.$$

Противоположное событие:

$$P(\bar{A}) = 1 - P(A).$$

Если событие A является частью события B, то есть  $A \subset B$ , то:

$$P(A) \leq P(B).$$

Это часто используется в тестах. Например, если известно  $P(X \geq 5) = 0.1$ , то  $P(X \geq 7) < 0.1$ , потому что событие  $X \geq 7$  уже, чем  $X \geq 5$ .

## 2.2 1.2. Сложение и умножение вероятностей

Для любых событий:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Если события несовместны, то  $A \cap B = \varnothing$ , значит:

$$P(A \cup B) = P(A) + P(B).$$

Условная вероятность:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Отсюда:

$$P(A \cap B) = P(A | B)P(B).$$

Если события независимы:

$$P(A \cap B) = P(A)P(B).$$

Важно: независимость не означает несовместность. Несовместные события при положительных вероятностях как раз зависимы.

## 2.3 1.3. Формула полной вероятности и Байес

Пусть события  $H_1, \dots, H_n$  образуют полную группу гипотез: они несовместны и покрывают все возможные случаи. Тогда:

$$P(A) = \sum_{i=1}^n P(A | H_i)P(H_i).$$

Формула Байеса:

$$P(H_k | A) = \frac{P(A | H_k)P(H_k)}{\sum_{i=1}^n P(A | H_i)P(H_i)}.$$

Типовая логика задачи на Байеса:

1. Выписать гипотезы  $H_i$ .
2. Выписать априорные вероятности  $P(H_i)$ .
3. Выписать условные вероятности  $P(A | H_i)$ .
4. Посчитать полную вероятность события A.
5. Подставить в Байеса.

## 2.4 1.4. Математическое ожидание и дисперсия

Математическое ожидание - среднее значение случайной величины в долгосрочной перспективе.

Для дискретной случайной величины:

$$E(X) = \sum_i x_i p_i.$$

Дисперсия:

$$D(X) = E[(X - E(X))^2].$$

Удобная формула:

$$D(X) = E(X^2) - [E(X)]^2.$$

Стандартное отклонение:

$$\sigma(X) = \sqrt{D(X)}.$$

Свойства:

$$E(aX + b) = aE(X) + b.$$

$$D(aX + b) = a^2 D(X).$$

Если X и Y независимы:

$$D(X + Y) = D(X) + D(Y).$$

## 3 2. Основные распределения

### 3.1 2.1. Бернулли

$X \sim \text{Bernoulli}(p)$  принимает 1 с вероятностью  $p$  и 0 с вероятностью  $1-p$ .

$$E(X) = p.$$

$$D(X) = p(1 - p).$$

Пример: один звонок успешен с вероятностью  $p=0.02$ . Успех/неуспех одной попытки - Бернулли.

### 3.2 2.2. Биномиальное распределение

$X \sim \text{Bin}(n, p)$  - число успехов в  $n$  независимых испытаниях Бернулли.

$$P(X = k) = C_n^k p^k (1 - p)^{n-k}.$$

$$E(X) = np.$$

$$D(X) = np(1 - p).$$

Типовые слова-маркеры: «из 10 попыток», «количество успехов», «каждая попытка независима», «вероятность успеха  $p$ ».

### 3.3 2.3. Геометрическое распределение

$X \sim \text{Geom}(p)$  - номер попытки, на которой впервые произошел успех.

$$P(X = k) = (1 - p)^{k-1}p.$$

$$E(X) = \frac{1}{p}.$$

Если вероятность дозвониться равна 0.02, то средний номер успешной попытки:

$$E(X) = \frac{1}{0.02} = 50.$$

Если человек дозвонился с 10-й попытки, он дозвонился на  $50 - 10 = 40$  попыток раньше среднего.

### 3.4 2.4. Экспоненциальное распределение

$X \sim \text{Exp}(\lambda)$  часто описывает время ожидания до события.

Плотность:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Функция распределения:

$$F(x) = P(X \leq x) = 1 - e^{-\lambda x}.$$

Хвостовая вероятность:

$$P(X \geq x) = e^{-\lambda x}.$$

Среднее:

$$E(X) = \frac{1}{\lambda}.$$

Дисперсия:

$$D(X) = \frac{1}{\lambda^2}.$$

Важное свойство: экспоненциальное распределение имеет бесконечный хвост. Нельзя утверждать, что событие точно произойдет не позже некоторого времени, например «каждый клиент придет не позднее чем через 8 минут». Вероятность после 8 минут может быть маленькой, но не нулевой.

Экзамнационный прием: если известно  $P(X \leq 3) = 0.6$ , то:

$$P(X \geq 3) = 1 - 0.6 = 0.4.$$

Значит:

$$P(X \geq 4) < P(X \geq 3) = 0.4 < 0.5.$$

Если известно  $P(X \geq 5) = 0.1$ , то:

$$P(X \geq 7) < P(X \geq 5) = 0.1.$$

### 3.5 2.5. Нормальное распределение

$X \sim N(\mu, \sigma^2)$  - нормальное распределение со средним  $\mu$  и дисперсией  $\sigma^2$ .

Стандартизация:

$$Z = \frac{X - \mu}{\sigma}.$$

После стандартизации:

$$Z \sim N(0, 1).$$

Если нужно найти  $P(X \leq a)$ , сначала переводим в  $Z$ :

$$P(X \leq a) = P\left(Z \leq \frac{a - \mu}{\sigma}\right).$$

### 3.6 2.6. Как пользоваться таблицей стандартного нормального распределения

Во многих демо-заданиях таблица показывает не  $P(Z \leq z)$ , а площадь между 0 и  $z$ :

$$P(0 \leq Z \leq z).$$

Для положительного  $z$ :

$$P(Z \leq z) = 0.5 + P(0 \leq Z \leq z).$$

$$P(Z \geq z) = 0.5 - P(0 \leq Z \leq z).$$

Для отрицательного  $z$  используется симметрия:

$$P(Z \leq -z) = 0.5 - P(0 \leq Z \leq z).$$

$$P(Z \geq -z) = 0.5 + P(0 \leq Z \leq z).$$

Пример: если в таблице для  $z=1.25$  стоит 0.3944, то:

$$P(Z \leq 1.25) = 0.5 + 0.3944 = 0.8944.$$

$$P(Z \geq 1.25) = 0.5 - 0.3944 = 0.1056.$$

Интервал между двумя положительными числами:

$$P(a \leq Z \leq b) = P(0 \leq Z \leq b) - P(0 \leq Z \leq a).$$

Интервал через ноль:

$$P(-a \leq Z \leq b) = P(0 \leq Z \leq a) + P(0 \leq Z \leq b).$$

## 4 3. Статистика

### 4.1 3.1. Генеральная совокупность и выборка

Генеральная совокупность - все объекты, о которых мы хотим сделать вывод. Выборка - часть генеральной совокупности, которую реально наблюдаем.

Параметр - характеристика генеральной совокупности. Например, истинное среднее  $\mu$ .

Статистика - характеристика выборки. Например, выборочное среднее  $\bar{x}$ .

### 4.2 3.2. Типы переменных

| Тип                        | Что означает                          | Примеры                             |
|----------------------------|---------------------------------------|-------------------------------------|
| Количественная непрерывная | может принимать вещественные значения | рост, доход, температура            |
| Количественная дискретная  | счетное количество                    | число покупок, число детей          |
| Категориальная номинальная | категории без порядка                 | город, пол, цвет                    |
| Категориальная порядковая  | категории с порядком                  | уровень образования, оценка сервиса |
| Бинарная                   | два значения                          | да/нет, 0/1                         |

Перед обучением модели категориальные признаки обычно нужно закодировать в числа.

### 4.3 3.3. Меры центра и разброса

Среднее:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Медиана - значение, которое делит упорядоченную выборку пополам. Медиана устойчивее среднего к выбросам.

Мода - самое частое значение.

Размах:

$$R = x_{max} - x_{min}.$$

Выборочная дисперсия:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Стандартное отклонение:

$$s = \sqrt{s^2}.$$

Межквартильный размах:

$$IQR = Q_3 - Q_1.$$

Типовое правило выбросов:

$$x < Q_1 - 1.5IQR \quad \text{или} \quad x > Q_3 + 1.5IQR.$$

## 4.4 3.4. Частотные таблицы и распределения

Абсолютная частота - сколько раз встречается значение.

Относительная частота - доля наблюдений:

$$\frac{\text{число объектов в категории}}{\text{общее число объектов}}.$$

Кумулятивная частота - накопленная частота до данного значения.

Гистограмма показывает распределение числового признака. Столбчатая диаграмма обычно показывает частоты категорий.

## 5 4. Проверка гипотез

### 5.1 4.1. Нулевая и альтернативная гипотезы

Нулевая гипотеза  $H_0$  - базовое предположение, обычно «эффекта нет», «разницы нет», «метод не влияет».

Альтернативная гипотеза  $H_1$  - то, что исследователь хочет обнаружить: «эффект есть», «метод улучшает», «средние отличаются».

Пример:

- $H_0$ : новый метод не влияет на успеваемость.
- $H_1$ : новый метод улучшает успеваемость.

### 5.2 4.2. p-value и уровень значимости

Уровень значимости  $\alpha$  - порог для принятия решения. Часто  $\alpha = 0.05$ .

p-value - вероятность получить такие же или более экстремальные данные при условии, что  $H_0$  верна.

Правило:

- если p-value  $\leq \alpha$ , отвергаем  $H_0$ ;
- если p-value  $> \alpha$ , не отвергаем  $H_0$ .

Важно: «не отвергаем  $H_0$ » не значит «доказали  $H_0$ ». Это значит: данных недостаточно, чтобы отвергнуть  $H_0$ .

Пример:  $\alpha=0.05$ , p-value=0.08. Так как  $0.08 > 0.05$ , мы не отвергаем  $H_0$ .

### 5.3 4.3. Ошибки первого и второго рода

| Решение           | Реальность  | Ошибка         |
|-------------------|-------------|----------------|
| Отвергли $H_0$    | $H_0$ верна | Ошибка I рода  |
| Не отвергли $H_0$ | $H_0$ ложна | Ошибка II рода |

Ошибка I рода: нашли эффект, которого на самом деле нет.

Ошибка II рода: не нашли эффект, который на самом деле есть.

Если в задаче p-value  $> \alpha$ , то  $H_0$  не отвергли. Поэтому возможная ошибка - ошибка второго рода.

Мощность критерия:

$$1 - \beta,$$

где  $\beta$  - вероятность ошибки второго рода. Мощность - вероятность обнаружить эффект, если он действительно есть.

## 6 5. Корреляция

### 6.1 5.1. Ковариация и корреляция

Ковариация показывает совместное изменение двух переменных. Если одна растет вместе с другой, ковариация положительная. Если одна растет, а другая уменьшается, ковариация отрицательная.

Коэффициент корреляции Пирсона:

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}.$$

Он лежит в диапазоне:

$$-1 \leq r \leq 1.$$

Интерпретация:

| Значение $r$ | Интерпретация                        |
|--------------|--------------------------------------|
| близко к 1   | сильная положительная линейная связь |
| близко к -1  | сильная отрицательная линейная связь |
| около 0      | линейной связи почти нет             |

Корреляция не означает причинность. Если доход и число покупок коррелируют, это не доказывает, что доход напрямую вызывает покупки.

### 6.2 5.2. Типовые ловушки

1. Корреляция Пирсона измеряет линейную связь. Нелинейная зависимость может иметь корреляцию около нуля.
2. Выбросы могут сильно исказить корреляцию.
3. Корреляция не задает направление причинности.
4. Для категориальных признаков обычная корреляция Пирсона часто некорректна без кодирования и осмысленной интерпретации.

## 7 6. Визуализация

### 7.1 6.1. Как выбрать график

| Задача                           | Подходящий график                       |
|----------------------------------|-----------------------------------------|
| Распределение числового признака | histogram, KDE, boxplot, violinplot     |
| Сравнить категории по значению   | barplot, boxplot by category            |
| Связь двух числовых признаков    | scatterplot                             |
| Связь трех признаков             | bubble plot: x, y, размер пузырька      |
| Динамика во времени              | line plot                               |
| Доли частей целого               | pie chart, stacked bar; осторожно с pie |
| Матрица корреляций               | heatmap                                 |

### 7.2 6.2. Как читать boxplot

Boxplot показывает:

- медиану;
- первый квартиль Q1;
- третий квартиль Q3;
- межквартильный размах  $IQR = Q3 - Q1$ ;
- усы;

- возможные выбросы.

Если медиана ближе к нижней границе коробки, распределение может быть скошено вправо. Если один ус сильно длиннее другого, это тоже признак асимметрии.

## 7.3 6.3. Как читать violinplot

Violinplot показывает форму распределения через оценку плотности. Широкое место означает, что там больше наблюдений. Он полезен, когда нужно сравнить не только медиану, но и форму распределения по группам.

## 7.4 6.4. Как читать density и CDF

Плотность  $f(x)$  сама по себе не является вероятностью в точке. Вероятность на интервале - это площадь под плотностью:

$$P(a \leq X \leq b) = \int_a^b f(x)dx.$$

Функция распределения:

$$F(x) = P(X \leq x).$$

Вероятность попасть в интервал:

$$P(a \leq X \leq b) = F(b) - F(a).$$

Хвост:

$$P(X \geq a) = 1 - F(a).$$

# 8 7. Работа с таблицами в pandas

## 8.1 7.1. Первичный осмотр данных

```
import pandas as pd

# загрузка
df = pd.read_csv("data.csv")

# первые строки
df.head()

# размер таблицы: строки, столбцы
df.shape

# информация о типах и пропусках
df.info()

# описательная статистика по числовым столбцам
df.describe()

# описательная статистика по всем столбцам
df.describe(include="all")
```

Полезные команды:

```

# названия столбцов
df.columns

# типы столбцов
df.dtypes

# число пропусков по столбцам
df.isna().sum()

# доля пропусков по столбцам
df.isna().mean()

# число уникальных значений
df.nunique()

# частоты категорий
df["city"].value_counts()

```

## 8.2 7.2. Фильтрация и выбор столбцов

```

# один столбец
df["age"]

# несколько столбцов
df[["age", "income"]]

# строки по условию
df[df["age"] >= 18]

# несколько условий: нужны скобки
df[(df["age"] >= 18) & (df["income"] > 50000)]

# или
df[(df["city"] == "Moscow") | (df["city"] == "SPb")]

```

## 8.3 7.3. Создание и изменение столбцов

```

# новый столбец
df["income_per_age"] = df["income"] / df["age"]

# преобразование по правилу
df["is_adult"] = (df["age"] >= 18).astype(int)

# применение функции
df["name_len"] = df["name"].str.len()

# замена значений
df["gender"] = df["gender"].replace({"M": "male", "F": "female"})

```

## 8.4 7.4. Пропущенные значения

```

# удалить строки с любыми пропусками
df_drop = df.dropna()

# заполнить числовой столбец медианой
df["age"] = df["age"].fillna(df["age"].median())

# заполнить категориальный столбец модой
mode_value = df["city"].mode()[0]

```

```
df["city"] = df["city"].fillna(mode_value)

# заполнить отдельным значением
df["education"] = df["education"].fillna("unknown")
```

Практическое правило:

- числовые признаки часто заполняют медианой или средним;
- категориальные признаки часто заполняют модой или отдельной категорией unknown;
- удаление строк допустимо, если пропусков мало и это не искажает выборку.

## 8.5 7.5. Изменение типов

```
# строку в число
df["income"] = pd.to_numeric(df["income"], errors="coerce")

# в категорию
df["city"] = df["city"].astype("category")

# в дату
df["date"] = pd.to_datetime(df["date"])

# извлечь год из даты
df["year"] = df["date"].dt.year
```

## 8.6 7.6. Агрегации

```
# средний доход по городам
df.groupby("city")["income"].mean()

# несколько агрегатов
df.groupby("city")["income"].agg(["mean", "median", "count"])

# агрегация нескольких столбцов
df.groupby("city").agg({
    "income": "mean",
    "age": "median",
    "client_id": "count"
})

# сводная таблица
pd.pivot_table(
    df,
    values="income",
    index="city",
    columns="education",
    aggfunc="mean"
)
```

## 8.7 7.7. Корреляции в pandas

```
# корреляционная матрица по числовым столбцам
df.corr(numeric_only=True)

# корреляция двух столбцов
df["age"].corr(df["income"])
```

Если спрашивают «между какими признаками максимальная корреляция», нужно убрать диагональ, потому что корреляция столбца с самим собой равна 1.

```
corr = df.corr(numeric_only=True).abs()

# занулить диагональ
import numpy as np
np.fill_diagonal(corr.values, 0)

corr.stack().idxmax(), corr.stack().max()
```

## 9 8. Машинное обучение: базовая схема

### 9.1 8.1. Объекты, признаки, целевая переменная

Объект - одна строка таблицы. Например, один клиент.

Признаки X - входные переменные: возраст, город, образование, доход.

Целевая переменная y - то, что нужно предсказать. Например, количество купленных ручек.

Если нужно предсказать количество, это регрессия. Если нужно предсказать категорию, это классификация.

Пример: есть поля «год покупки», «возраст», «пол», «уровень образования», «средний доход», «город». Нужно спрогнозировать количество купленных ручек. Целевая переменная - количество купленных ручек. Задача - регрессия.

### 9.2 8.2. Обучение с учителем и без учителя

Обучение с учителем: есть правильные ответы y.

- регрессия: y - число;
- классификация: y - класс.

Обучение без учителя: правильных ответов нет.

- кластеризация;
- поиск структуры;
- снижение размерности.

### 9.3 8.3. Train/test split

Нельзя оценивать качество модели только на тех данных, на которых она обучалась. Нужна отложенная выборка.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Для классификации полезно сохранять баланс классов:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

## 10 9. Метрики качества

### 10.1 9.1. Метрики регрессии

Ошибка на объекте:

$$e_i = y_i - \hat{y}_i.$$

Абсолютная ошибка:

$$|y_i - \hat{y}_i|.$$

MAE:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MSE:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

RMSE:

$$RMSE = \sqrt{MSE}.$$

Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Интерпретация:

| Метрика | Когда подходит                           | Особенность              |
|---------|------------------------------------------|--------------------------|
| MAE     | нужна средняя ошибка в исходных единицах | устойчивее к выбросам    |
| MSE     | нужно сильнее штрафовать большие ошибки  | чувствительна к выбросам |
| RMSE    | как MSE, но в исходных единицах          | крупные ошибки важнее    |
| R2      | сравнить с константной моделью           | может быть отрицательным |

Если в вариантах есть только classification-метрики и MAE, а задача про предсказание числа, выбирай MAE.

Если завышение и занижение имеют разную стоимость, нужна асимметричная функция потерь. Но если среди вариантов ее нет, выбираем корректную регрессионную метрику из списка.

## 10.2 9.2. Confusion matrix

Для бинарной классификации:

|                   | Предсказан positive | Предсказан negative |
|-------------------|---------------------|---------------------|
| Истинный positive | TP                  | FN                  |
| Истинный negative | FP                  | TN                  |

TP - верно предсказали положительный класс.

TN - верно предсказали отрицательный класс.

FP - ложноположительный ответ.

FN - ложноотрицательный ответ.

## 10.3 9.3. Accuracy

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Accuracy показывает долю правильных ответов.

Ловушка: если классы несбалансированы, accuracy может быть бесполезной. Если 90% объектов класса 0, модель, которая всегда предсказывает 0, получит accuracy 0.9.

Поэтому по accuracy 0.7 на train и 0.65 на test нельзя однозначно сказать, хорошая модель или плохая, если неизвестны количество классов и баланс классов.

## 10.4 9.4. Precision, recall, F1

Precision:

$$precision = \frac{TP}{TP + FP}.$$

Ответ на вопрос: среди тех, кого модель назвала положительными, какая доля действительно положительная?

Recall:

$$recall = \frac{TP}{TP + FN}.$$

Ответ на вопрос: какую долю реальных положительных объектов модель нашла?

F1-score:

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}.$$

F1 полезен, когда важны и precision, и recall.

macro F1 - среднее F1 по классам без учета размера классов.

weighted F1 - среднее F1 по классам с весами по числу объектов в каждом классе.

## 10.5 9.5. ROC-AUC

ROC-AUC измеряет качество ранжирования объектов: насколько модель ставит положительные объекты выше отрицательных.

Вероятностная интерпретация:

$$ROC-AUC = P(a(x_+) > a(x_-)) + \frac{1}{2}P(a(x_+) = a(x_-)).$$

Если модель всем объектам выдает одну и ту же вероятность  $a(x)=\text{beta}$ , то для любой пары positive-negative оценки равны. Тогда:

$$ROC-AUC = 0 + \frac{1}{2} \cdot 1 = 0.5.$$

ROC-AUC = 0.5 соответствует случайному ранжированию. ROC-AUC = 1 - идеальное ранжирование.

## 10.6 9.6. Метрики в sklearn

```

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

```

*# регрессия*

```

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

```

*# классификация*

```

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc = roc_auc_score(y_test, y_pred_proba)
cm = confusion_matrix(y_test, y_pred)

```

Для roc\_auc\_score нужны не жесткие классы 0/1, а вероятности или scores:

```

y_pred_proba = model.predict_proba(X_test)[: , 1]
roc_auc_score(y_test, y_pred_proba)

```

## 11 10. Переобучение, недообучение, регуляризация

### 11.1 10.1. Недообучение

Модель слишком простая и плохо работает и на train, и на test.

Пример:

| Train | Test | Интерпретация                    |
|-------|------|----------------------------------|
| 0.55  | 0.54 | недообучение или слабые признаки |

Что делать:

- взять более сложную модель;
- добавить признаки;
- уменьшить регуляризацию;
- обучать дольше, если это итерационный алгоритм.

### 11.2 10.2. Переобучение

Модель слишком хорошо подстроилась под train и хуже работает на test.

Пример:

| Train | Test | Интерпретация        |
|-------|------|----------------------|
| 0.98  | 0.62 | сильное переобучение |

Что делать:

- усилить регуляризацию;
- уменьшить сложность модели;
- использовать кросс-валидацию;

- собрать больше данных;
- для деревьев ограничить глубину;
- для нейросетей применить dropout, weight decay, early stopping.

### 11.3 10.3. Когда нельзя сделать вывод

Если train accuracy 0.70, test accuracy 0.65, разница небольшая. Это не похоже на сильное переобучение. Но нельзя сказать, хорошее качество или плохое, не зная баланса классов.

Для классификации всегда спрашивай:

- сколько классов;
- какие доли классов;
- какая baseline-модель;
- какая метрика соответствует бизнес-цели.

### 11.4 10.4. Регуляризация

L2-регуляризация:

$$Q(w) + \lambda \sum_j w_j^2.$$

L1-регуляризация:

$$Q(w) + \lambda \sum_j |w_j|.$$

L2 уменьшает веса, делает модель более гладкой.

L1 может занулять некоторые веса и выполнять отбор признаков.

lambda или alpha - сила регуляризации. Чем она больше, тем проще модель и тем выше риск недообучения.

## 12 11. Кросс-валидация и подбор гиперпараметров

### 12.1 11.1. Зачем нужна кросс-валидация

Один train/test split может оказаться случайно удачным или неудачным. Кросс-валидация делит данные на несколько частей и оценивает модель несколько раз.

K-fold cross-validation:

1. Данные делятся на K частей.
2. Модель K раз обучается на K-1 частях.
3. Оставшаяся часть используется для проверки.
4. Итоговая метрика усредняется.

### 12.2 11.2. GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_leaf": [1, 5]
}
```

```

grid = GridSearchCV(
    model,
    param_grid=param_grid,
    scoring="f1",
    cv=5,
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_
best_params = grid.best_params_
best_score = grid.best_score_

```

Важная ловушка: test set нельзя использовать для выбора гиперпараметров. Его используют только для финальной оценки.

## 13 12. Линейная алгебра для анализа данных и ML

### 13.1 12.1. Векторы и матрицы

Вектор признаков объекта:

$$x = (x_1, x_2, \dots, x_d).$$

Матрица признаков:

$$X \in \mathbb{R}^{n \times d},$$

где  $n$  - число объектов,  $d$  - число признаков.

Вектор весов линейной модели:

$$w \in \mathbb{R}^d.$$

Линейное предсказание:

$$\hat{y} = x^T w + b.$$

Для всех объектов:

$$\hat{y} = Xw + b.$$

### 13.2 12.2. Скалярное произведение

$$x^T y = \sum_{i=1}^d x_i y_i.$$

Скалярное произведение связано с углом:

$$x^T y = \|x\| \|y\| \cos \theta.$$

Если скалярное произведение положительное, угол острый. Если отрицательное, угол тупой. Если равно нулю, векторы ортогональны.

### 13.3 12.3. Норма

Евклидова норма:

$$\|x\|_2 = \sqrt{\sum_i x_i^2}.$$

Манхэттенская норма:

$$\|x\|_1 = \sum_i |x_i|.$$

В kNN и кластеризации расстояние между объектами часто считается через нормы.

### 13.4 12.4. Умножение матриц

Если:

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times k},$$

то:

$$AB \in \mathbb{R}^{m \times k}.$$

Число столбцов первой матрицы должно совпадать с числом строк второй.

В общем случае:

$$AB \neq BA.$$

### 13.5 12.5. Обратная матрица и ранг

Обратная матрица  $A^{-1}$  существует только для квадратной невырожденной матрицы.

$$AA^{-1} = A^{-1}A = I.$$

Ранг матрицы - максимальное число линейно независимых строк или столбцов.

Если признаки линейно зависимы, возникают проблемы мультиколлинеарности в линейных моделях.

## 14 13. Линейные модели и градиентный спуск

### 14.1 13.1. Линейная регрессия

Модель:

$$\hat{y} = w_0 + w_1x_1 + \dots + w_dx_d.$$

Часто минимизируется MSE:

$$Q(w) = \frac{1}{n} \sum_{i=1}^n (y_i - x_i^T w)^2.$$

## 14.2 13.2. Логистическая регрессия

Используется для классификации. Для бинарной классификации:

$$p(y = 1|x) = \sigma(x^T w),$$

где сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Если вероятность выше порога, например 0.5, предсказывается класс 1.

## 14.3 13.3. Градиентный спуск

Общее обновление:

$$w_{k+1} = w_k - \eta \nabla Q(w_k).$$

eta - learning rate, размер шага.

Если eta слишком большой, алгоритм может расходиться. Если слишком маленький, обучение будет медленным.

## 14.4 13.4. Momentum

Momentum накапливает направление движения:

$$v_{k+1} = \rho v_k + \nabla Q(w_k),$$

$$w_{k+1} = w_k - \eta v_{k+1}.$$

Momentum помогает сгладить скачки и быстрее двигаться вдоль устойчивого направления.

## 14.5 13.5. AdaGrad и разные шаги для разных весов

AdaGrad накапливает квадраты градиентов по координатам:

$$v_k = v_{k-1} + (\nabla Q(w_k))^2.$$

Обновление:

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{v_k} + \varepsilon} \nabla Q(w_k).$$

Здесь  $v_k$  - вектор. Поэтому эффективный шаг для координаты  $i$ :

$$\frac{\eta}{\sqrt{v_{k,i}} + \varepsilon}.$$

Он может быть разным для разных весов. Если по координате часто были большие градиенты, шаг уменьшается.

Обычный градиентный спуск использует один общий eta для всех координат.

## 15 14. Классические алгоритмы ML

### 15.1 14.1. kNN

k-nearest neighbors предсказывает ответ по ближайшим объектам обучающей выборки.

Для классификации: класс определяется голосованием соседей.

Для регрессии: предсказывается среднее значение соседей.

Гиперпараметры:

- n\_neighbors;
- метрика расстояния;
- веса соседей.

Малое k - низкое смещение, высокий разброс, риск переобучения.

Большое k - выше смещение, ниже разброс, риск недообучения.

kNN чувствителен к масштабу признаков, поэтому часто нужна стандартизация.

### 15.2 14.2. Решающие деревья

Дерево рекурсивно делит пространство признаков на области.

Плюсы:

- умеет ловить нелинейности;
- работает с признаками разных типов после кодирования;
- интерпретируемо при небольшой глубине;
- не требует обязательного масштабирования.

Минусы:

- легко переобучается;
- нестабильно: небольшое изменение данных может сильно изменить дерево.

Основные гиперпараметры:

- max\_depth;
- min\_samples\_split;
- min\_samples\_leaf;
- criterion.

### 15.3 14.3. Bagging и Random Forest

Bagging обучает много моделей на разных bootstrap-выборках и усредняет ответы.

Random Forest - ансамбль деревьев, где каждое дерево обучается на bootstrap-выборке и при разбиениях рассматривает случайное подмножество признаков.

Главный эффект случайного леса: уменьшение разброса.

Если к решающему лесу добавить еще одно дерево:

- разброс обычно уменьшится;
- шум не изменится;
- смещение в базовой теории почти не изменится.

Шум - свойство данных, модель его не убирает.

### 15.4 14.4. Boosting

Boosting строит модели последовательно. Каждая следующая модель исправляет ошибки предыдущих.

Идея:

- много слабых моделей;
- последовательное улучшение;
- высокая точность при грамотной настройке.

Примеры: Gradient Boosting, XGBoost, LightGBM, CatBoost.

Boosting часто уменьшает смещение, но при слишком большой сложности может переобучиться.

## 15.5 14.5. Bias-variance-noise decomposition

Ожидаемая ошибка модели может быть представлена как сумма трех частей:

$$error = bias^2 + variance + noise.$$

Смещение (bias) - систематическая ошибка модели. Высокий bias означает, что модель слишком простая.

Разброс (variance) - чувствительность модели к обучающей выборке. Высокий variance означает, что модель нестабильна и склонна к переобучению.

Шум (noise) - неустраняемая случайность в данных.

Типовая картина:

| Модель               | Bias                          | Variance                 |
|----------------------|-------------------------------|--------------------------|
| Очень простая модель | высокий                       | низкий                   |
| Глубокое дерево      | низкий                        | высокий                  |
| Random Forest        | примерно тот же bias деревьев | ниже variance            |
| Boosting             | часто снижает bias            | может увеличить variance |

## 16 15. Кластеризация

### 16.1 15.1. K-means

K-means делит объекты на k кластеров так, чтобы минимизировать сумму квадратов расстояний до центроидов.

Алгоритм:

1. Выбрать k центров.
2. Назначить каждый объект ближайшему центру.
3. Пересчитать центры как средние объектов кластера.
4. Повторять до стабилизации.

Плюсы:

- простой и быстрый;
- хорошо работает для компактных округлых кластеров.

Минусы:

- нужно заранее задать k;
- чувствителен к масштабу признаков;
- чувствителен к выбросам;
- плохо работает для кластеров сложной формы.

### 16.2 15.2. DBSCAN

DBSCAN ищет плотные области.

Основные параметры:

- eps - радиус окрестности;
- min\_samples - минимальное число соседей для ядровой точки.

Типы точек:

- core point - достаточно соседей;
- border point - соседей мало, но находится рядом с core point;
- noise point - шумовая точка, не относится к кластерам.

Плюсы:

- не нужно заранее задавать число кластеров;
- умеет находить кластеры сложной формы;
- умеет выделять шум.

Минусы:

- чувствителен к  $\epsilon$ ;
- плохо работает при сильно разной плотности кластеров;
- требует масштабирования признаков.

## 16.3 15.3. Метрики кластеризации

Silhouette score:

$$s = \frac{b - a}{\max(a, b)},$$

где  $a$  - среднее расстояние до объектов своего кластера,  $b$  - среднее расстояние до ближайшего другого кластера.

Диапазон:

$$-1 \leq s \leq 1.$$

Чем ближе к 1, тем лучше.

Если есть истинные метки классов, можно использовать ARI или NMI. Если истинных меток нет, используют внутренние метрики, например silhouette.

## 17 16. Тексты

### 17.1 16.1. Токенизация

Токенизация - разбиение текста на токены: слова, подслова, символы или знаки препинания.

Пример:

```
"Кошки спят на диване" -> ["Кошки", "спят", "на", "диване"]
```

### 17.2 16.2. Стоп-слова

Стоп-слова - часто встречающиеся служебные слова, которые иногда удаляют: «и», «в», «на», «это».

Удаление стоп-слов полезно не всегда. Например, в задачах анализа тональности отрицание «не» важно.

### 17.3 16.3. Стемминг и лемматизация

Стемминг грубо отрезает окончание и оставляет основу/стем.

```
кошками -> кошк  
бегущий -> бегущ
```

Лемматизация приводит слово к нормальной словарной форме.

```
кошками -> кошка  
бегущие -> бегущий  
купил, купила, купили -> купить
```

Лемматизация нужна, чтобы снизить число разных словоформ. После лемматизации словарь обычно меньше, признаков меньше, риск переобучения обычно ниже.

Неверно: «после лемматизации количество разных токенов увеличивается». Обычно оно уменьшается.

Неверно: «лемматизация оставляет только основу». Это ближе к стеммингу.

## 17.4 16.4. Bag of Words и TF-IDF

Bag of Words представляет текст как вектор частот слов.

TF-IDF учитывает частоту слова в документе и редкость слова в корпусе.

Идея:

- слово часто встречается в данном документе - вес выше;
- слово встречается почти во всех документах - вес ниже.

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

texts = ["кошки любят молоко", "собаки любят мясо"]

bow = CountVectorizer()
X_bow = bow.fit_transform(texts)

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(texts)
```

## 18 17. Изображения

### 18.1 17.1. Представление изображений

Черно-белое изображение можно представить как матрицу размера:

$$H \times W.$$

Цветное RGB-изображение можно представить как тензор:

$$H \times W \times 3.$$

Где H - высота, W - ширина, 3 - каналы Red, Green, Blue.

Пиксели часто имеют значения от 0 до 255. Нормализация переводит их, например, в диапазон [0,1]:

```
X = X / 255.0
```

### 18.2 17.2. Зачем нужна нормализация изображений

Нормализация:

- ускоряет обучение;
- делает значения признаков сопоставимыми;
- помогает численной стабильности;
- снижает риск проблем с градиентами.

### 18.3 17.3. Базовые операции

Типовые преобразования:

- resize - изменение размера;
- crop - обрезка;
- grayscale - перевод в оттенки серого;
- normalization - масштабирование пикселей;
- augmentation - случайные повороты, сдвиги, отражения.

Аугментация помогает бороться с переобучением, потому что модель видит больше вариантов похожих изображений.

## 19 18. Основы нейронных сетей

### 19.1 18.1. Нейрон и слой

Искусственный нейрон считает линейную комбинацию входов и применяет функцию активации:

$$h = \phi(w^T x + b).$$

Слой - набор нейронов, применяемых к входу.

### 19.2 18.2. Dense layer

Полносвязный слой:

$$h = \phi(Wx + b).$$

Каждый вход связан с каждым нейроном слоя.

### 19.3 18.3. Convolutional layer

Сверточный слой использует фильтры, которые скользят по изображению и извлекают локальные признаки.

Плюсы CNN для изображений:

- учитывает локальную структуру;
- использует одни и те же фильтры в разных местах;
- имеет меньше параметров, чем полносвязная сеть на пикселях.

### 19.4 18.4. Pooling

Pooling уменьшает пространственный размер карты признаков.

Max pooling берет максимум в окне. Average pooling берет среднее.

Плюсы pooling:

- уменьшает число параметров/вычислений;
- делает модель менее чувствительной к небольшим сдвигам.

### 19.5 18.5. Dropout

Dropout во время обучения случайно зануляет часть нейронов. Это регуляризация, которая снижает переобучение.

Во время предсказания dropout обычно выключен.

### 19.6 18.6. Batch normalization

BatchNorm нормализует активации внутри mini-batch. Обычно помогает ускорить и стабилизировать обучение.

### 19.7 18.7. Функции активации

| Активация  | Формула/идея                            | Где используется             |
|------------|-----------------------------------------|------------------------------|
| Sigmoid    | $1 / (1 + \exp(-x))$                    | бинарная вероятность         |
| Tanh       | значения от -1 до 1                     | скрытые слои, реже сейчас    |
| ReLU       | $\max(0, x)$                            | стандарт для скрытых слоев   |
| Leaky ReLU | ReLU с малым наклоном слева             | борьба с dying ReLU          |
| Softmax    | превращает logits в вероятности классов | многоклассовая классификация |

Softmax:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

## 20 19. Кодирование категориальных признаков и sklearn Pipeline

### 20.1 19.1. One-hot encoding

Категориальные признаки нельзя напрямую подавать во многие модели как строки. Их кодируют.

```
pd.get_dummies(df, columns=["city", "education"], drop_first=False)
```

Для sklearn-пайплайна лучше использовать OneHotEncoder.

### 20.2 19.2. Полный шаблон для задачи классификации

```
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score

# y - целевая переменная, X - признаки
X = df.drop(columns=["target"])
y = df["target"]

num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols)
])

model = Pipeline([
    ("prep", preprocess),
    ("clf", RandomForestClassifier(random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```

)

model.fit(X_train, y_train)

pred = model.predict(X_test)
proba = model.predict_proba(X_test)[:, 1]

print("accuracy", accuracy_score(y_test, pred))
print("f1", f1_score(y_test, pred))
print("roc_auc", roc_auc_score(y_test, proba))

```

## 20.3 19.3. Полный шаблон для задачи регрессии

```

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X = df.drop(columns=["target"])
y = df["target"]

num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols)
])

model = Pipeline([
    ("prep", preprocess),
    ("reg", RandomForestRegressor(random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("MAE", mean_absolute_error(y_test, pred))
print("MSE", mean_squared_error(y_test, pred))
print("R2", r2_score(y_test, pred))

```

## 20.4 19.4. GridSearchCV с Pipeline

```
param_grid = {
    "reg_n_estimators": [100, 200],
    "reg_max_depth": [None, 5, 10],
    "reg_min_samples_leaf": [1, 3, 5]
}

grid = GridSearchCV(
    model,
    param_grid=param_grid,
    scoring="neg_mean_absolute_error",
    cv=5,
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_
pred = best_model.predict(X_test)
```

Для классификатора шаг называется `clf`, поэтому параметры будут выглядеть как `clf__max_depth`, `clf__n_estimators`.

## 21 20. Быстрые экзаменационные правила

### 21.1 20.1. Как распознать целевую переменную

Целевая переменная - то, что нужно предсказать.

- «Предсказать количество покупок» ->  $y$  = количество покупок.
- «Предсказать уйдет ли клиент» ->  $y$  = уйдет/не уйдет.
- «Предсказать цену квартиры» ->  $y$  = цена.
- «Разделить клиентов на группы» -> целевой переменной нет, кластеризация.

Клиент, студент, товар, объект - обычно не целевая переменная, а объект наблюдения.

### 21.2 20.2. Как распознать метрику

- Предсказываем число -> регрессионная метрика: MAE/MSE/RMSE/R2.
- Предсказываем класс -> classification metric: accuracy/precision/recall/F1/ROC-AUC.
- Дисбаланс классов -> accuracy может обманывать.
- Важны ложные срабатывания -> precision.
- Важно найти как можно больше positives -> recall.
- Нужен баланс precision и recall -> F1.
- Важна ранжировка вероятностей -> ROC-AUC.

### 21.3 20.3. Как распознать ошибку в гипотезах

1. Сравни  $p$ -value и  $\alpha$ .
2. Если  $p \leq \alpha$ ,  $H_0$  отвергли.
3. Если  $p > \alpha$ ,  $H_0$  не отвергли.
4. Отвергли истинную  $H_0$  -> ошибка I рода.
5. Не отвергли ложную  $H_0$  -> ошибка II рода.

### 21.4 20.4. Как распознать переобучение

Сравни train и test.

- train высокий, test намного ниже -> переобучение;
- train и test оба низкие -> недообучение;
- train и test близкие -> сильного переобучения нет;
- качество хорошее или плохое нельзя понять без baseline и контекста.

## 21.5 20.5. Как думать про интервалы вероятности

Если событие уже, его вероятность меньше.

- $X \geq 7$  уже, чем  $X \geq 5$ , значит  $P(X \geq 7) \leq P(X \geq 5)$ .
- $X \leq 1$  уже, чем  $X \leq 3$ , значит  $P(X \leq 1) \leq P(X \leq 3)$ .
- Для непрерывного распределения  $P(X = a) = 0$ , поэтому обычно нет разницы между  $<$  и  $\leq$ .

## 21.6 20.6. Мини-таблица «что выбрать»

| Вопрос                          | Ответ                                     |
|---------------------------------|-------------------------------------------|
| Предсказать количество          | Регрессия                                 |
| Предсказать класс               | Классификация                             |
| Нет target                      | Обучение без учителя                      |
| Всем объектам одинаковый score  | ROC-AUC = 0.5                             |
| p-value 0.08, alpha 0.05        | $H_0$ не отвергаем                        |
| $H_0$ не отвергли, но она ложна | Ошибка II рода                            |
| Лемматизация                    | нормальная словарная форма                |
| Стемминг                        | грубая основа слова                       |
| Добавить дерево в Random Forest | variance уменьшается                      |
| Noise при изменении модели      | не меняется                               |
| Train 0.70, test 0.65           | нет сильного переобучения, нужен контекст |
| Разные шаги по координатам GD   | AdaGrad/RMSProp/Adam-подобные методы      |

## 22 21. Мини-задачи для самопроверки

### 22.1 21.1. Геометрическое распределение

Вероятность успеха одной попытки  $p = 0.04$ . Сколько попыток в среднем нужно до первого успеха?

Решение:

$$E(X) = \frac{1}{p} = \frac{1}{0.04} = 25.$$

### 22.2 21.2. p-value

$H_0$ : метод не влияет.  $\alpha = 0.05$ ,  $p\text{-value} = 0.03$ . Что делаем?

Решение:  $0.03 \leq 0.05$ , значит отвергаем  $H_0$ . Возможная ошибка - ошибка I рода.

### 22.3 21.3. Accuracy и дисбаланс

В выборке 95% объектов класса 0. Модель всегда предсказывает 0. Accuracy?

Решение: 0.95. Но модель бесполезна для поиска класса 1.

### 22.4 21.4. ROC-AUC константного алгоритма

Алгоритм всем объектам выдает вероятность 0.2. ROC-AUC?

Решение: все scores равны, значит ROC-AUC = 0.5.

### 22.5 21.5. Нормальная таблица

В таблице для  $z = 1.00$  дана площадь от 0 до  $z$ : 0.3413. Найти  $P(Z \geq 1)$ .

Решение:

$$P(Z \geq 1) = 0.5 - 0.3413 = 0.1587.$$

## 22.6 21.6. Экспоненциальное распределение

Известно  $P(X \geq 5) = 0.1$ . Что можно сказать про  $P(X \geq 8)$ ?

Решение:  $X \geq 8$  - более узкое событие, значит  $P(X \geq 8) < 0.1$ .

## 23 22. Финальный чек-лист перед сдачей

Перед ответом на вопрос проверь:

1. Что является объектом?
2. Что является признаками?
3. Есть ли целевая переменная?
4. Если target есть: это число или класс?
5. Какая метрика соответствует типу задачи?
6. Нет ли дисбаланса классов?
7. Сравниваются ли train и test?
8. Не путаются ли probability density и probability?
9. Не путаются ли лемматизация и стемминг?
10. Не используется ли test set для подбора гиперпараметров?
11. Для таблицы нормального распределения: таблица дает  $P(Z \leq z)$  или площадь между 0 и  $z$ ?
12. Для p-value: сначала сравнить с alpha, потом говорить об ошибках.

## 24 23. Краткий словарь

baseline - простая модель для сравнения, например всегда предсказывать среднее или самый частый класс.

feature - признак, входная переменная.

target - целевая переменная, то, что предсказываем.

loss function - функция потерь, которую оптимизирует алгоритм.

metric - метрика качества, по которой оцениваем результат.

hyperparameter - параметр, который задается до обучения: глубина дерева, число соседей, learning rate.

parameter - параметр, который модель выучила: веса линейной модели, пороги дерева.

regularization - ограничение сложности модели для борьбы с переобучением.

cross-validation - многократная проверка модели на разных разбиениях train/validation.

overfitting - переобучение.

underfitting - недообучение.

token - единица текста после токенизации.

lemma - нормальная словарная форма слова.

embedding - векторное представление объекта, например слова или изображения.

epoch - один проход по обучающей выборке при обучении нейросети.

batch - группа объектов, обрабатываемых за один шаг обучения.